Ledgerline interest calculation: wiki pages and I/O contract

Wiki pages generated by the FlowX.AI legacy transformation agent from the Ledgerline batch application. CONTRACT.md is the input/output contract for the build.

Pages included:

- `02-processes/process-interest-calculation.md`
- `10-acceptance-criteria/acceptance-criteria-interest-calculation.md`
- `01-data-model/project-data-model.md`
- `CONTRACT.md`

`02-processes/process-interest-calculation.md`

Process: Interest Calculation (Batch Pipeline)

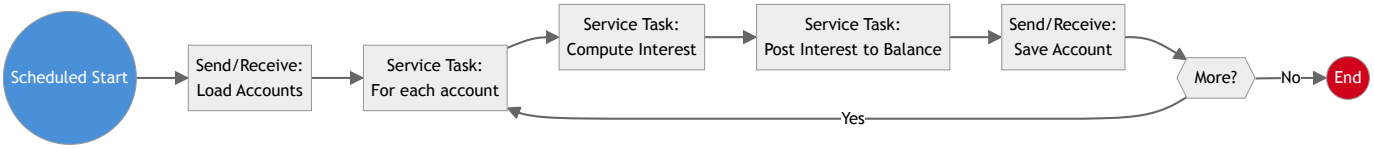
Powered by FLOWX.AI Coding Agents

Field	Value
Name	Interest Calculation
Type	Scheduled Process (orchestration)
Legacy Source	`LXB002C` (+ `LXB0001–03C`) via `ACCRUE.jcl`; disclosure config `RATEGRP.jcl`
Status	Draft
Related Data	Account, TransactionCategoryBalance

Business Description

Periodic pipeline that computes interest for each account based on its balance and disclosure/interest group, then posts the interest to the account. Replaces the `LXB002C/ACCRUE` batch. The interest-rate parameters come from the disclosure-group reference data (`RATEGRP`).

Process Flow



Nodes

#	Node	Type	Description	Legacy Reference
1	Scheduled Start	Start Event (Scheduled Trigger)	Interest cycle	`ACCRUE.jcl`
2	Load Accounts	Send/Receive Message Task	Read accounts	`LXB001C` read
3	For each account	Service Task (loop)	Iterate	main loop
4	Compute Interest	Service Task (NoOpHandler)	`interest = balance × rate(group)`	`LXB002C`
5	Post Interest	Service Task (NoOpHandler)	Add interest to balance	balance update
6	Save Account	Send/Receive Message Task	Persist	`REWRITE LEDGFILE`

Business Rules

Rule: ComputeInterest

- **Legacy source:** `LXB002C` interest computation using disclosure-group rate.
- **Logic:** `interest = currentBalance × interestRate(groupId)`; posted to balance and category balance.
- **FLOWX implementation:** NoOpHandler; interest rate from disclosure-group reference data / Configuration Parameter.
- `{{TODO: Extract exact interest formula, rounding, and grouping from LXB002C + RATEGRP.}}`

Integration Touchpoints

Send Node	Topic	Payload	Receive	System
Load Accounts	`ai.flowx.in.ledgerline.datastore.loadaccounts`	`{ page }`	Account[]	FlowX Persistence
Save Account	`ai.flowx.in.ledgerline.datastore.saveaccount`	Account	ack	FlowX Persistence

Swimlanes

- **System**

---

*Powered by **FLOWX.AI** Coding Agents*

Acceptance Criteria: Interest Calculation

> **Powered by FLOWX.AI Coding Agents** · `@regression`

Verifies Interest Calculation (`LXB002C`/ACCRUE).

Configuration Dependencies

Config	Effect
Disclosure-group rates (`RATEGRP`)	Interest rate per account group

Scenarios

#	Scenario	Steps	Expected Result
1	Compute interest (happy)	Run over accounts	Interest = balance × group rate; matches legacy
2	Rounding (boundary)	Balance causing fractional cents	Rounding matches legacy `{{TODO: confirm rounding mode}}`
3	Zero balance (edge)	Account balance 0	Zero interest
4	Multiple disclosure groups (gateway)	Accounts in different groups	Correct rate per group

Data Mapping Verification

Flow	Verify
Compute Interest	balance × rate(groupId) → posted interest

Legacy Comparison

Input	**Legacy** `LXB002C`	**FLOWX**
account set	interest amounts	identical per account

Project Data Model — Ledgerline → FLOWX.AI

> **Powered by FLOWX.AI Coding Agents**

This section documents all **Project Data Types** derived from the Ledgerline copybooks (`/app/cpy`). Each VSAM record layout maps to a Project Data Type; COBOL `05`-level items become attributes. Packed/zoned/signed numerics (`COMP-3`, `S9(n)V99`) map to `NUMBER`, fixed-width `X(10)` dates map to `DATE`, and FILLER tails are dropped. Enumerated reference codes are documented in Enumerations.

> **Type-conversion note:** `S9(10)V99` → `NUMBER` (decimal, scale 2); `9(n)` → `NUMBER` (integer); `X(n)` → `STRING` (maxLength n); `X(10)` date → `DATE`; `X(26)` timestamp → `DATE` (timestamp). The `LACC-EXPIRY-DATE` legacy typo is corrected to `expirationDate` — see `{{DECISION}}` below.

Business Categories

Category	Types
Core Processing	Account, Transaction, TransactionCategoryBalance
Domain	Customer, Card, CardXref
Reference / Config	TransactionType, TransactionCategory
Security	SecurityUser
Navigation (internal)	SessionContext

Account

Replaces the ACCOUNT-RECORD VSAM KSDS (`LEDGFILE` / `LXV022Y`, 300 bytes). Key: `accountId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
accountId	NUMBER	Yes	11 digits, unique key	Account identifier	`LACC-ID 9(11)`
activeStatus	ENUM	Yes	AccountStatus	Active status flag	`LACC-ACTIVE-STATUS X(1)`
currentBalance	NUMBER	Yes	decimal(12,2), signed	Current account balance	`LACC-CURR-BAL S9(10)V99`
creditLimit	NUMBER	Yes	decimal(12,2), signed	Credit limit	`LACC-CREDIT-LIMIT S9(10)V99`
cashCreditLimit	NUMBER	Yes	decimal(12,2), signed	Cash credit limit	`LACC-CASH-CREDIT-LIMIT S9(10)V99`
openDate	DATE	Yes		Account open date	`LACC-OPEN-DATE X(10)`
expirationDate	DATE	Yes		Account expiration date	`LACC-EXPIRY-DATE X(10)` *(legacy typo)*
reissueDate	DATE	No		Account reissue date	`LACC-REISSUE-DATE X(10)`

Attribute	Type	Required	Constraints	Description	Legacy Source
currentCycleCredit	NUMBER	Yes	decimal(12,2), signed	Current-cycle credits	`LACC-CURR-CYC-CREDIT S9(10)V99`
currentCycleDebit	NUMBER	Yes	decimal(12,2), signed	Current-cycle debits	`LACC-CURR-CYC-DEBIT S9(10)V99`
addressZip	STRING	No	max 10	Account ZIP code	`LACC-ADDR-ZIP X(10)`
groupId	STRING	No	max 10	Disclosure/interest group id	`LACC-GROUP-ID X(10)`

`{{DECISION: Correct the legacy field name typo LACC-EXPIRY-DATE → expirationDate (recommended), or preserve the misspelling for round-trip fidelity? ETL must map the legacy column either way.}}`

Used by: Account View, Account Update, Bill Payment, Transaction Posting, Interest Calculation.

Customer

Replaces CUSTOMER-RECORD (`CLNTFILE` / `LXV025Y`, 500 bytes). Key: `customerId`.

Attribute	Type	Required	Constraints	Description	Legacy Source
customerId	NUMBER	Yes	9 digits, unique key	Customer identifier	`CLNT-ID 9(9)`
firstName	STRING	Yes	max 25	First name	`CLNT-FIRST-NAME X(25)`
middleName	STRING	No	max 25	Middle name	`CLNT-MIDDLE-NAME X(25)`
lastName	STRING	Yes	max 25	Last name	`CLNT-LAST-NAME X(25)`
addressLine1	STRING	Yes	max 50	Address line 1	`CLNT-ADDR-LINE-1 X(50)`
addressLine2	STRING	No	max 50	Address line 2	`CLNT-ADDR-LINE-2 X(50)`
addressLine3	STRING	No	max 50	Address line 3	`CLNT-ADDR-LINE-3 X(50)`
addressStateCode	STRING	No	max 2	State code	`CLNT-ADDR-STATE-CD X(2)`
addressCountryCode	STRING	No	max 3	Country code	`CLNT-ADDR-COUNTRY-CD X(3)`
addressZip	STRING	No	max 10	ZIP code	`CLNT-ADDR-ZIP X(10)`
phoneNumber1	STRING	No	max 15	Primary phone	`CLNT-PHONE-NUM-1 X(15)`
phoneNumber2	STRING	No	max 15	Secondary phone	`CLNT-PHONE-NUM-2 X(15)`
ssn	NUMBER	Yes	9 digits (PII)	Social security number	`CLNT-SSN 9(9)`
govtIssuedId	STRING	No	max 20	Government-issued id	`CLNT-GOVT-ISSUED-ID X(20)`
dateOfBirth	DATE	No		Date of birth	`CLNT-DOB-YYYY-MM-DD X(10)`
eftAccountId	STRING	No	max 10	EFT account id	`CLNT-EFT-ACCOUNT-ID X(10)`
primaryCardHolderInd	STRING	No	max 1	Primary-holder indicator	`CLNT-PRI-CRD-HOLDER-IND X(1)`
ficoCreditScore	NUMBER	No	300–850	FICO credit score	`CLNT-FICO-CREDIT-SCORE 9(3)`

{{TODO: SSN and govt id are PII — define masking/encryption and access restrictions in FLOWX.}}

Used by: Account View, Account Update, Authorization.

Card

Replaces CRD-RECORD (CRDFILE + CRDAIX / LXV023Y, 150 bytes). Key: cardNumber; alternate index on cardAccountId.

Attribute	Type	Required	Constraints	Description	Legacy Source
cardNumber	STRING	Yes	16 chars, unique key (PCI)	Card number	CRD-NUM X(16)
cardAccountId	NUMBER	Yes	11 digits, AIX	Owning account id	CRD-LACC-ID 9(11)
cvvCode	NUMBER	Yes	3 digits (PCI)	Card verification value	CRD-CVV-CD 9(3)
embossedName	STRING	Yes	max 50	Name embossed on card	CRD-EMBOSSED-NAME X(50)
expirationDate	DATE	Yes		Card expiration date	CRD-EXPIRY-DATE X(10) *(legacy typo)*
activeStatus	ENUM	Yes	CardStatus	Card active status	CRD-ACTIVE-STATUS X(1)

{{TODO: cardNumber and cvvCode are PCI-sensitive — define tokenization/masking strategy.}}

Used by: Card List, Card View, Card Update, Card Security, Authorization.

CardXref

Replaces CRD-LINK-RECORD (LINKFILE + LKACAIX / LXV024Y, 50 bytes). Links card ↔ customer ↔ account; alternate index on accountId.

Attribute	Type	Required	Constraints	Description	Legacy Source
cardNumber	STRING	Yes	16 chars, key	Card number	LINK-CRD-NUM X(16)
customerId	NUMBER	Yes	9 digits	Customer id	LINK-CLNT-ID 9(9)
accountId	NUMBER	Yes	11 digits, AIX	Account id	LINK-LACC-ID 9(11)

Used by: Account View, Card List, Transaction Add, Authorization.

Transaction

Replaces TXN-RECORD (TXNFILEX / LXV030Y, 350 bytes). Key: transactionId.

Attribute	Type	Required	Constraints	Description	Legacy Source
transactionId	STRING	Yes	16 chars, unique key	Transaction identifier	TXN-ID X(16)

Attribute	Type	Required	Constraints	Description	Legacy Source
typeCode	ENUM	Yes	TransactionType	Transaction type code	`TXN-TYPE-CD X(2)`
categoryCode	ENUM	Yes	TransactionCategory	Transaction category code	`TXN-CAT-CD 9(4)`
source	STRING	No	max 10	Transaction source	`TXN-SOURCE X(10)`
description	STRING	No	max 100	Transaction description	`TXN-DESC X(100)`
amount	NUMBER	Yes	decimal(11,2), signed	Transaction amount	`TXN-AMT S9(9)V99`
merchantId	NUMBER	No	9 digits	Merchant id	`TXN-MERCHANT-ID 9(9)`
merchantName	STRING	No	max 50	Merchant name	`TXN-MERCHANT-NAME X(50)`
merchantCity	STRING	No	max 50	Merchant city	`TXN-MERCHANT-CITY X(50)`
merchantZip	STRING	No	max 10	Merchant ZIP	`TXN-MERCHANT-ZIP X(10)`
cardNumber	STRING	Yes	16 chars	Card used	`TXN-CRD-NUM X(16)`
originTimestamp	DATE	No	timestamp	Origination timestamp	`TXN-ORIG-TS X(26)`
processTimestamp	DATE	No	timestamp	Processing timestamp	`TXN-PROC-TS X(26)`

Used by: Transaction List, Transaction View, Transaction Add, Transaction Posting, Reports.

TransactionCategoryBalance

Replaces TXN-CAT-BAL-RECORD (`BALCATF` / `LXV026Y`, 50 bytes). Composite key: `accountId` + `typeCode` + `categoryCode`.

Attribute	Type	Required	Constraints	Description	Legacy Source
accountId	NUMBER	Yes	11 digits, key part	Account id	`BCAT-LACC-ID 9(11)`
typeCode	ENUM	Yes	TransactionType, key part	Transaction type	`BCAT-TYPE-CD X(2)`
categoryCode	ENUM	Yes	TransactionCategory, key part	Category	`BCAT-CD 9(4)`
balance	NUMBER	Yes	decimal(11,2), signed	Category running balance	`TXN-CAT-BAL S9(9)V99`

Used by: Transaction Posting, Interest Calculation, Reports.

SecurityUser

Replaces OPR-USER-DATA (`OPRSEC` / `LXS020Y`). Key: `userId`. Target FLOWX model gates access with roles; plaintext password is replaced by IdP credentials.

Attribute	Type	Required	Constraints	Description	Legacy Source
userId	STRING	Yes	8 chars, unique key	User id	`OPR-OP-ID X(8)`

Attribute	Type	Required	Constraints	Description	Legacy Source
firstName	STRING	Yes	max 20	First name	`OPR-OP-FNAME X(20)`
lastName	STRING	Yes	max 20	Last name	`OPR-OP-LNAME X(20)`
password	STRING	—	**DO NOT MIGRATE**	Plaintext password (legacy debt)	`OPR-OP-PWD X(8)`
userType	ENUM	Yes	UserType	Admin (`A`) / User (`U`)	`OPR-OP-TYPE X(1)`

`{{DECISION: Do NOT migrate the plaintext password column. Provision users in a modern IdP and map userType → FLOWX roles. Confirm IdP (Keycloak/other).}}`

Used by: Signon, User Administration.

SessionContext

Internal process data replacing the navigation portion of `LEDGERLINE-COMMAREA` (`LX0014Y`). Not persisted — lives in process instance data. Documented here for traceability; in FLOWX most of these become implicit process state.

Attribute	Type	Description	Legacy Source
userId	STRING	Signed-on user id	`LLINE-USER-ID X(8)`
userType	ENUM	UserType	`LLINE-USER-TYPE X(1)` (88 A/U)
selectedCustomerId	NUMBER	Current customer context	`LLINE-CLNT-ID 9(9)`
selectedAccountId	NUMBER	Current account context	`LLINE-LACC-ID 9(11)`
selectedCardNumber	STRING	Current card context	`LLINE-CRD-NUM 9(16)`

`{{DECISION: The from/to tranid/program and last-map/mapset fields (LLINE-FROM-*, LLINE-TO-*, LLINE-LAST-MAP) are pure navigation plumbing — replace with FLOWX process transitions and do not model them as data. Confirm.}}`

See Data Model Diagram for relationships.

---

*Powered by **FLOWX.AI** Coding Agents*

Candidate I/O contract

What a migrated implementation must produce so the scorer can compare it with the oracle. The contract fixes **formats only**. It says nothing about behaviour, so it may be handed to a builder without leaking the answers.

Inputs (given per run)

The same files the COBOL chain reads, one record per line, fixed width, layouts as in the copybooks:

File	Copybook	Record
`daytxn.dat`	LXV031Y	350
`ledgacc.dat`	LXV022Y	300
`cardlink.dat`	LXV024Y	50
`balcat.dat`	LXV026Y	50
`rategrp.dat`	LXV027Y	50
`clntrec.dat`, `crdrec.dat`, `txntype.dat`, `txncatg.dat`	LXU021C, LXV023Y, LXV028Y, LXV029Y	500, 150, 60, 60

Run parameters: business date `2022-07-18`, ACCRUE parameter `2022071800`.

Step isolation. A candidate for ACCRUE receives the oracle's POSTDAY outputs as its account and category-balance inputs, so an ACCRUE result is never penalised for a POSTDAY error.

Outputs

One JSON file per output: a list of objects, in any order. Field names and types are fixed. Strings may omit trailing spaces. Amounts are JSON strings or numbers holding at most two decimals (`"14.56"`, `-500`).

File	Step	Fields
`postday_txnfile.json`	POSTDAY	transaction fields (below) for every posted transaction
`postday_dayrej.json`	POSTDAY	transaction fields + `reason` (int) + `reason_desc` for every rejected transaction
`postday_ledgacc.json`	POSTDAY	account fields for every account, after posting
`postday_balcat.json`	POSTDAY	`acct_id, type_cd, cat_cd, bal` for every category balance, after posting
`accrue_systxn.json`	ACCRUE	transaction fields for every interest transaction written
`accrue_ledgacc.json`	ACCRUE	account fields for every account, after interest

- Transaction fields: `tran_id, type_cd, cat_cd (int), source, desc, amt, merchant_id (int), merchant_name, merchant_city, merchant_zip, card_num, orig_ts, proc_ts`
- Account fields: `acct_id (int), active_status, curr_bal, credit_limit, cash_credit_limit, open_date, expiration_date, reissue_date, curr_cyc_credit, curr_cyc_debit, addr_zip, group_id`

A run may instead leave the oracle's fixed-width dumps (`<name>.txt`). The scorer reads either form.

Optional `flags.json`: the uncertainties the pipeline declared **before** scoring, as `[{"rule": "R-INT-TRUNC", "source": "<artefact path and line>"}]` or `[{"output": "...", "key": "...", "source": "..."}]`. See PROTOCOL.md §6 for what counts as a flag.